

0.1 Finite Volume Method

The finite volume method (FVM) is a widely used discretization method for various problems governed by conservation laws. The FVM is particularly robust and well-suited for arbitrary complex geometry. A key advantage of FVM that benefits this simulation is its natural enforcement of local mass conservation.

To begin with, we consider a general hyperbolic conservation law regarding a scalar variable u as

$$\begin{cases} \frac{\partial u(\mathbf{x}, t)}{\partial t} + \nabla \cdot \mathbf{f}(u; \mathbf{x}, t) = 0, & t > 0, \mathbf{x} \in \Omega, \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega, \end{cases} \quad (1)$$

where we assume the domain $\Omega \subset \mathbb{R}^2$. The term $\mathbf{f}$ represents the flux transporting the scalar variable u . The basic semi-discretized formulation of the finite volume method can be written as

$$\frac{\partial \bar{u}_E}{\partial t} + \frac{1}{|E|} \int_{\partial E} \mathbf{F}(\mathbf{x}, t) \cdot \boldsymbol{\nu} d\mathbf{x} = 0, \quad (2)$$

where the cell-averaged value $\bar{u}$ is calculated as

$$\bar{u}_E = \frac{1}{|E|} \int_E u(\mathbf{x}, t) d\mathbf{x}, \quad (3)$$

where $|E|$ denote the area of the cell E . Here, $\mathbf{F}(\mathbf{x}, t)$ denotes the approximated numerical flux. We employ the Lax-Friedrichs numerical flux

$$\mathbf{F}(u^+, u^-) = \frac{1}{2} \left[(\mathbf{f}(u^+) + \mathbf{f}(u^-)) \cdot \boldsymbol{\nu} - \alpha_{LF}(u^+ - u^-) \right], \quad (4)$$

where u^+ and u^- denote the values approximated from the perspective of element $E^+ \in \mathcal{T}_h$ and $E^- \in \mathcal{T}_h$ respectively, where $E^+ \cap E^-$ is an edge of the mesh. The unit normal vector $\boldsymbol{\nu}$ points from E^- to E^+ , and is orthogonal to the edge. The parameter α_{LF} is the Lax-Friedrichs stabilizer factor. Depending on the available information about the solution, there are two possible approaches for computing α_{LF} :

- The global Lax-Friedrichs Stabilizer:

$$\alpha_{LF} = \max_{\mathbf{x} \in \Omega, t > 0, u \in \mathbb{R}} \left| \frac{\partial \mathbf{f}(u; \mathbf{x}, t) \cdot \boldsymbol{\nu}}{\partial u} \right|;$$

- The local Lax-Friedrichs Stabilizer:

$$\alpha_{LF} = \max_{\text{edges } e} \left(\left| \frac{\partial \mathbf{f}(u^+; \mathbf{x}, t) \cdot \boldsymbol{\nu}}{\partial u} \right|_e, \left| \frac{\partial \mathbf{f}(u^-; \mathbf{x}, t) \cdot \boldsymbol{\nu}}{\partial u} \right|_e \right).$$

The numerical flux $\mathbf{F}$, as defined in equation (4), is consistent and conservative across element interfaces. We now replace the u^+ and u^- with reconstructed values $\mathcal{R}^+$ and $\mathcal{R}^-$ and express the numerical flux as

$$\mathbf{F}(\mathcal{R}^+, \mathcal{R}^-) = \frac{1}{2}[(\mathbf{f}(\mathcal{R}^+) + \mathbf{f}(\mathcal{R}^-) \cdot \boldsymbol{\nu} - \alpha_{LF}(\mathcal{R}^+ - \mathcal{R}^-)]. \quad (5)$$

Here, the reconstructed value $\mathcal{R}$ is of higher-order accuracy. We evaluate the integrals over each edge $e_i \in \partial E$ with a suitable quadrature rule. Let $|e_i|$ denote the length of edge e_i . Assuming a quadrature rule with G point per edge, the semi-discretized formulation becomes:

$$\frac{\partial \bar{u}_E}{\partial t} + \sum_{i=0}^4 \frac{|e_i|}{|E|} \sum_{g=0}^G \omega_{i,g} \mathbf{F}(\mathcal{R}_{i,g}^+, \mathcal{R}_{i,g}^-) \cdot \boldsymbol{\nu}_i = 0, \quad (6)$$

where $\omega_{i,g}$ represents the weight associated to the g^{th} quadrature points on edge e_i .

0.2 The Multi-level Weighted Essentially Non-Oscillatory (ML-WENO) Method

We developed an accurate, versatile, robust and efficient finite volume multi level weighted essentially non oscillatory (ML-WENO) reconstruction in Arbogast et al. (2024). ML-WENO is primarily used to reconstruct values on both sides of the element interfaces. It can also be applied to a general reconstruction of discontinuous variables, e.g., piece-wise constant discontinuous Darcy pressure.

0.2.1 Stencil Polynomials

Consider rectangular stencils $\mathcal{S} = \{E_{i,j}, i \leq s, j \leq t\} \subset \mathcal{T}_h$ on a quasi uniform logically rectangular mesh, which covers the area $\Omega_{\mathcal{S}} = \cup_{E \in \mathcal{S}} E$. For each stencil, we construct a tensor product stencil polynomial $Q_{s,t}^{\mathcal{S}} \in \mathbb{Q}_{s-1,t-1}(\Omega_{\mathcal{S}})$. The scaled tensor product stencil polynomial is expressed as

$$Q_{s,t}^{\mathcal{S}}(x, y) = \sum_{p < s, q < t} c_{p,q} \left(\frac{x - x_{\mathcal{S}}}{h_{\mathcal{S}}} \right)^p \left(\frac{y - y_{\mathcal{S}}}{h_{\mathcal{S}}} \right)^q, \quad (7)$$

where $(x_{\mathcal{S}}, y_{\mathcal{S}})$ is any point in the stencil $\mathcal{S}$. We choose the stencil “center” point: the intersection point of two diagonal lines joining top left, bottom right and top right, bottom left corners. Assuming that each cell $E_{i,j} \in \mathcal{S} \subset \mathcal{T}_h$ is shape regular as in Definition ??, we define the stencil scale parameter $h_{\mathcal{S}} = \sum_{i \leq s, j \leq t} E_{i,j} / (st)$ as the average diameter of all cells in the stencil. Introducing the normalized coordinates

$$\xi = \frac{x - x_{\mathcal{S}}}{h_{\mathcal{S}}}, \quad \text{and} \quad \eta = \frac{y - y_{\mathcal{S}}}{h_{\mathcal{S}}}. \quad (8)$$

We can rewrite equation (7) in normalized coordinates as

$$\hat{Q}_{s,t}^s(\xi, \eta) = \sum_{p < s, q < t} c_{p,q} \xi^p \eta^q. \quad (9)$$

The tensor product stencil polynomial satisfies the condition that the cell-averaged value (3) is preserved through integration over each cell as

$$\frac{1}{|E_{i,j}|} \iint_{E_{i,j}} Q_{s,t}^s(x, y) dx dy = \bar{u}_{i,j}, \quad \forall E_{i,j} \in \mathcal{S}, \quad (10)$$

where $\bar{u}_{E_{i,j}}$ represents the cell-averaged scalar variable over cell $E_{i,j}$.

One way to construct a stencil polynomial $Q_{s,t}^s$ is using basis polynomials. Let $Q_{s,t}^s = \sum_{i,j} \bar{u}_{i,j} \phi_{s,t}^{i,j}$, where the basis polynomials $\phi_{s,t}^{i,j}$ satisfy

$$\frac{1}{|E_{i',j'}|} \iint_{E_{i',j'}} \phi_{s,t}^{i,j}(x, y) dx dy = \begin{cases} 1 & (i', j') = (i, j) \\ 0 & (i', j') \neq (i, j) \end{cases}, \quad \forall E_{i',j'} \in \mathcal{S}, \quad (11)$$

which always leads to an $(st) \times (st)$ non-singular linear system. We can write the basis polynomials in terms of normalized coordinates ξ and η as well, and rewrite them as

$$\hat{Q}_{s,t}^s(\xi, \eta) = \sum_{i,j} \bar{u}_{i,j} \hat{\phi}_{s,t}^{i,j}(\xi, \eta). \quad (12)$$

The approxmatibility of a tensor product polynomial is related to the associated complete polynomial, as discussed by Dupont and Scott (1980). Given a bounded interpolation projection operator $\mathcal{Q}$, for all $\mathbf{x} \in E \in \mathcal{S}$, we have

$$|D^\alpha u(\mathbf{x}) - D^\alpha(\mathcal{Q}u(\mathbf{x}))| \leq \begin{cases} Ch^{r-|\alpha|} & \text{if } u \text{ is smooth on the stencil,} \\ C & \text{otherwise,} \end{cases} \quad (13)$$

where $\alpha = (\alpha_1, \alpha_2)$ is a two-dimensional multi-index, and $|\alpha| = \alpha_1 + \alpha_2 \leq r$. This theorem applies to our stencil polynomials (7).

In most two-dimensional cases, we implement a square stencil with $s = t = r$. Uneven stencils ($s \neq t$) are used in the following two situations.

- When some a prior knowledge of the solution is acquired before simulation, e.g., the transport velocity is known to be fixed in a single direction.
- When derivatives are approximated, e.g., diffusive flux is computed across the element edge, where a stretched stencil is necessary to compensate for the loss of accuracy associated with derivative approximation.

For simplicity, we restrict the following discussions to the square case $s = t = r$.

0.2.2 Stencil Polynomial Smoothness Indicators

A smoothness indicator σ is defined to quantify the smoothness of the tensor product polynomial $Q_{r,r}^{\mathcal{S}}$ on the stencil $\mathcal{S}$. The classical approach by Jiang and Shu (1996) and Friedrich (1998) defines σ in analogue to a Sobolev seminorm as

$$\sigma_{JS} = \sum_{1 \leq |\alpha| \leq r} \int \int_{E_0} \frac{h_s^{2|\alpha|}}{|E_0|} (\mathcal{D}^\alpha Q(x, y))^2 dx dy, \quad (14)$$

where E_0 can be any cell $E \in \mathcal{S}$, and is usually referred to as the target cell, i.e., the cell E from which (x, y) is taken in $Q_{r,r}^{\mathcal{S}}(x, y)$.

The Jiang-Shu smoothness indicator can be computed utilizing the predefined basis polynomials (11):

$$\begin{aligned} \sigma_{JS} &= \sum_{1 \leq |\alpha| \leq r} \frac{h_s^{2|\alpha|}}{|E_0|} \iint_{E_0} (\mathcal{D}^\alpha Q(x, y))^2 dx dy \\ &= \frac{h_s^2}{|E_0|} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_0} (\mathcal{D}^\alpha \hat{Q}(\xi, \eta))^2 d\xi d\eta \\ &= \frac{h_s^2}{|E_0|} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_0} \left(\sum_{ij} \bar{u}_{ij} \mathcal{D}^\alpha \hat{\phi}_{i,j}(\xi, \eta) \right)^2 d\xi d\eta \\ &= \frac{h_s^2}{|E_0|} \sum_{ij} \sum_{i'j'} \bar{u}_{ij} \bar{u}_{i'j'} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_{ij}} \mathcal{D}^\alpha \hat{\phi}_{ij}(\xi, \eta) \mathcal{D}^\alpha \hat{\phi}_{i'j'}(\xi, \eta) d\xi d\eta \\ &= \frac{h_s^2}{|E_0|} \sum_{ij} \sum_{i'j'} \bar{u}_{ij} \bar{u}_{i'j'} \sigma_{ij}^{i'j'}, \end{aligned} \quad (15)$$

where the coefficient tensor $\sigma_{ij}^{i'j'}$ can be precomputed before the time-stepping loop, after the mesh is established.

Alternatively, we can compute the Jiang-Shu smoothness indicator directly using the polynomial coefficients formulation (9) as

$$\begin{aligned} \sigma_{JS} &= \frac{h_s^2}{|E_0|} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_0} (\mathcal{D}^\alpha \hat{Q}(\xi, \eta))^2 d\xi d\eta \\ &= \sum_{|\gamma| \leq r} \sum_{|\beta| \leq r} c_\gamma c_\beta \left(\sum_{1 \leq |\alpha| \leq r} \frac{h_s^2}{|E_0|} \iint_{E_0} \mathcal{D}^\alpha \xi^\gamma \mathcal{D}^\alpha \eta^\beta d\xi d\eta \right) \\ &= \sum_{|\gamma| \leq r} \sum_{|\beta| \leq r} c_\gamma c_\beta \zeta_{\gamma, \beta}, \end{aligned} \quad (16)$$

which paves the way for further simplification in (19) below.

The smoothness indicator exhibits the following convergence property: there exists a constant $C > 0$ such that when $h \rightarrow 0^+$,

$$\sigma = \begin{cases} Ch^2 + \mathcal{O}(h^3) & \text{u is smooth on } \Omega_s, \\ \Theta(1) & \text{u is not smooth on } \Omega_s. \end{cases} \quad (17)$$

Actually $\sigma = O(1)$ in the nonsmooth case, but in practice we find that $\sigma = \Theta(1)$. Being $\Theta(1)$ means that σ is bounded above and below by a constant.

0.2.3 Two Alternative Smoothness Indicators

We propose two alternative ways of defining a smoothness indicator

- *Simple smoothness indicator* Huang et al. (2023)

$$\sigma_{\text{simp}} = \frac{1}{N_s - 1} \sum_k (\bar{u}_k - \bar{u}_0)^2, \quad (18)$$

where $\bar{u}_0$ is the cell averaged solution in the target cell. This simple smoothness indicator is designed to reduce the computational cost and to deal with distorted stencils on unstructured meshes. However, this simple smoothness indicator is not as robust as the classic Jiang-Shu smoothness indicator.

- *Polynomial smoothness indicator* Arbogast et al. (2025)

$$\sigma_P = \sum_{\alpha} \zeta_{\alpha, \alpha} c_{\alpha}^2 \quad (19)$$

which is obtained by dropping all the cross terms appeared in the formulation (16). This alternative serves as a good approximation to the classic Jiang-Shu smoothness indicator while the computing efficiency is greatly improved.

0.2.4 ML-WENO weighting

Given a target cell $E_{i,j}$, we can create a nonempty set of stencils $\{\mathcal{S}_l \ni E_{i,j}, l = 0, 1, \dots, L, L \geq 0\}$. (See Figure 1 for an example.) The reconstruction of u on the target cell $E_{i,j}$ is a convex combination of the stencil polynomials defined in equation (12)

$$\mathcal{R}(\mathbf{x}) = \sum_l \tilde{\omega}_l Q^{\mathcal{S}_l}(\mathbf{x}), \quad \mathbf{x} \in E_{i,j}, \quad (20)$$

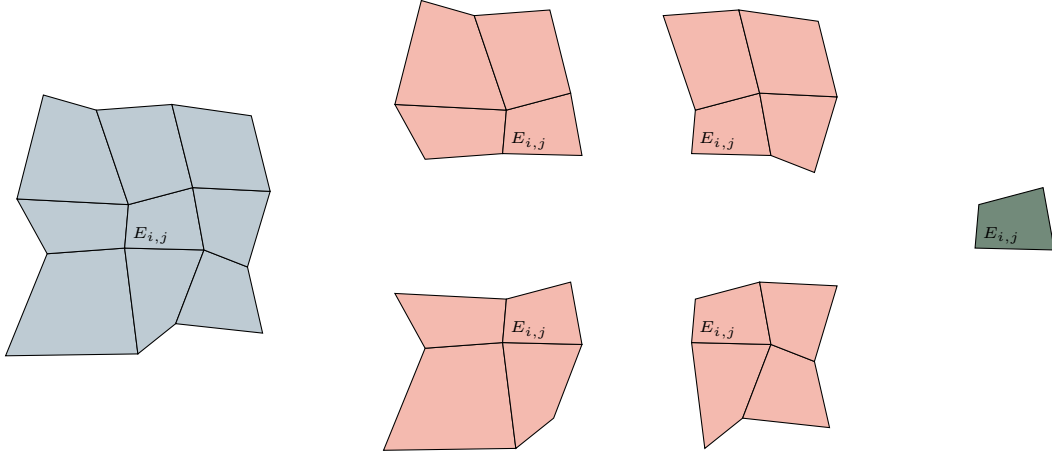


Figure 1: An example of a (3,2,1) multi-level stencils combination on a logically rectangular quadrilateral mesh.

where $\sum_l \tilde{\omega}_l = 1$ and $\tilde{\omega}_l$ are the normalized nonlinear weights associated with stencils. We begin by arbitrarily choosing a set of linear weights $\{\omega_l\}$ and scale them with stencil smoothness indicator as

$$\hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^{ar_l + n_l}}, \quad (21)$$

where $r_l = \max\{s, t\} = r$ in the cases of using square stencils, and the constants $\epsilon = 1e-4$ and $a > 0.5$ are chosen accordingly. The biasing factor $n_l = n(r_l)$ is used to biasing further away from the constant level, whose smoothness indicator is always zero. We choose

$$n(r_l) = n_l = \begin{cases} 1, & \text{if } r_l = 1, \\ 3, & \text{if } r_l = 2, \\ 4, & \text{if } r_l \geq 3. \end{cases} \quad (22)$$

We finally renormalise the scaled weights as

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}}. \quad (23)$$

0.2.5 The ML-WENO Reconstruction Error

In this section, we analyze the accuracy of the new multi-level weighting scheme. Consider the set containing indices of stencils $l \in \mathcal{L} = \{0, 1, 2, \dots, L\}$. Specifically, we denote the set of indices of stencils of size $s \times t$ as $\mathcal{L}_{s,t} \subset \mathcal{L}$. We assume the mesh resolution is sufficiently fine such that at least one smooth stencil exists in the

presence of shocks. Accordingly, we define two index sets: one containing the stencils identified as smooth, and the other containing those that exhibit discontinuities as

$$\begin{aligned}\text{Smooth} &= \{l : u \text{ is smooth on } \mathcal{S}_l\}, \\ \text{Jump} &= \{l : u \text{ has a jump on } \mathcal{S}_l\},\end{aligned}\tag{24}$$

where $\text{Smooth} \cup \text{Jump} = \mathcal{L}$ and $\text{Smooth} \cap \text{Jump} = \emptyset$. Moreover, we assume $\text{Smooth} \neq \emptyset$. We show that the reconstruction is accurate with respect to the target cell E_0 to order

$$r_{\max} = \max\{r : \text{Smooth} \cap \mathcal{L}_{r,r} \neq \emptyset\}.\tag{25}$$

Figure 2 illustrates a (3,2,1) combination of stencils in the presence of two shocks. In this example, the set containing indices of smooth stencils is $\text{Smooth} = \{0, 1\}$, indicating that u on stencils $\mathcal{S}_0$ and $\mathcal{S}_1$ does not have a jump. Consequently, we expect the accuracy of the reconstruction is $r_{\max} = \text{Smooth} \cap \mathcal{L}_{2,2} = 2$.

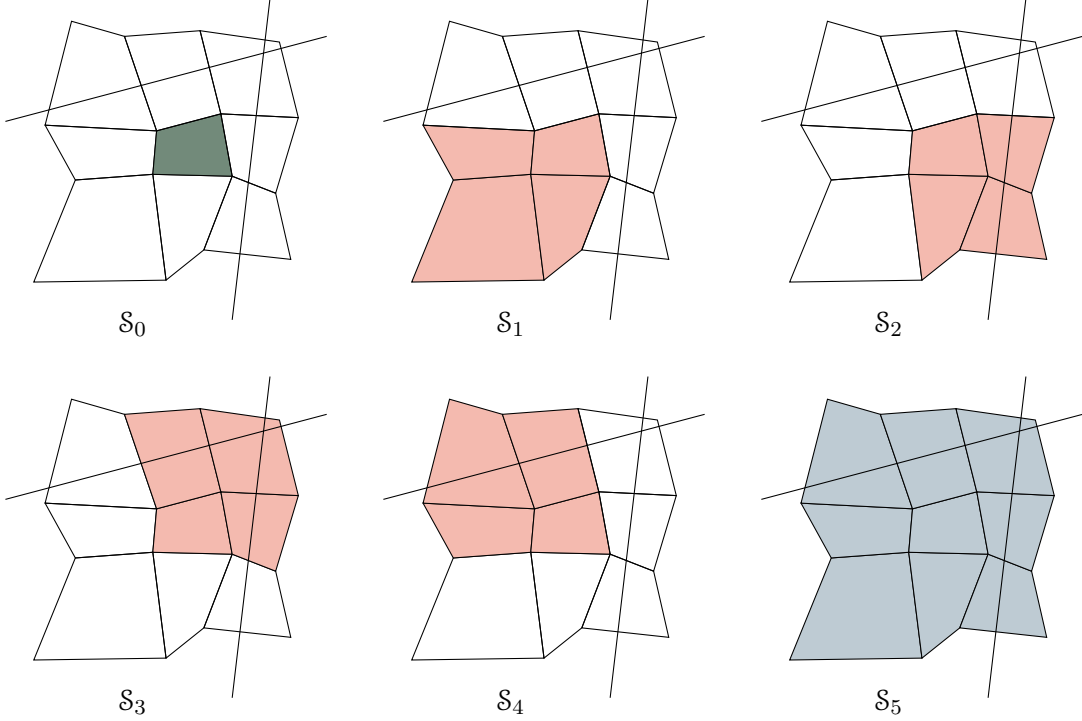


Figure 2: An illustration of a (3,2,1) multi-level stencils combination on a logically rectangular quadrilateral mesh with two shock present.

We follow the estimation of reconstruction error in Arbogast et al. (2024). We start by estimating the error of the nonlinear scaling (21) associated with some

stencils $\mathcal{S}_l$ for some $m > 0$.

$$\hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^m}. \quad (26)$$

We estimate the reconstruction error under two distinct situations:

- $l \in \text{Smooth}$, the smoothness indicator $\sigma_l = Ch^2 + \mathcal{O}(h^3)$;
- $l \in \text{Jump}$, the smoothness indicator $\sigma_l = \Theta(1)$.

We employ the asymptotic estimation result from Olver (1997). Assuming $n \geq m$, $a > 0$, and $b > 0$, then

$$\begin{aligned} \frac{a}{bh^m + \mathcal{O}(h^n)} &= \frac{ah^{-m}}{b}(1 + \mathcal{O}(h^{n-m})), \\ \frac{a}{bh^{-n} + \mathcal{O}(h^{-m})} &= \frac{ah^n}{b}(1 + \mathcal{O}(h^{n-m})). \end{aligned} \quad (27)$$

For the smooth stencil $\mathcal{S}_l$, $l \in \text{Smooth}$

$$\begin{aligned} \hat{\omega}_l &= \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^m} = \frac{\omega_l}{((C + \epsilon)h_0^2 + \mathcal{O}(h^3))^m} \\ &= \omega_l \left(\frac{h_0^{-2}}{C + \epsilon} (1 + \mathcal{O}(h)) \right)^m \\ &= \frac{\omega_l}{(D + \epsilon)^m} h_0^{-2m} (1 + \mathcal{O}(h)). \end{aligned} \quad (28)$$

In conclusion, we estimate the nonlinear scaling as

$$\hat{\omega}_l = \begin{cases} \frac{\omega_l}{(C + \epsilon)^m} h_0^{-2m} (1 + \mathcal{O}(h)), & \text{if } l \in \text{Smooth}, \\ \Theta(1), & \text{if } l \in \text{Jump}. \end{cases} \quad (29)$$

Then we estimate the error for the normalized nonlinear weighting (23). We expand the sum of nonlinear scaling as

$$\sum_l \hat{\omega}_l = \sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \hat{\omega}_l + \sum_{l \in \text{Jump}} \hat{\omega}_l. \quad (30)$$

We estimate the smooth part of the nonlinear weighting as

$$\sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \hat{\omega}_l = \sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \frac{\omega_l}{(C + \epsilon)^m} h_0^{-2(ar+\eta)} (1 + \mathcal{O}(h)). \quad (31)$$

Assume $h < 1$, we keep the largest $ar + \eta(r)$ number as our estimator

$$\begin{aligned}
\sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \hat{\omega}_l &= \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r_{\max}, r_{\max}}} \frac{\omega_l}{(C + \epsilon)^{ar_{\max} + \eta(r_{\max})}} h_0^{-2(ar_{\max} + \eta(r_{\max}))} (1 + \mathcal{O}(h)) \\
&\quad + \Theta(h^{-2(ar + \eta(r))}) \\
&= \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r_{\max}, r_{\max}}} \frac{\omega_l}{(C + \epsilon)^{ar_{\max} + \eta(r_{\max})}} h_0^{-2(ar_{\max} + \eta(r_{\max}))} \\
&\quad + \mathcal{O}(h^{1-2(ar_{\max} + \eta(r_{\max}))}).
\end{aligned} \tag{32}$$

later, we denote the set $\text{Smooth} \cap \mathcal{L}_{r,r}$ as Smooth_r for simplicity. The overall accuracy estimation of the nonlinear weighting (23) is presented as

- $l \in \text{Smooth}$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}} = \frac{\omega_l}{\sum_{l' \in \text{Smooth}_r} \omega_{l'}} h^{2[a(r_{\max} - r) + \eta(r_{\max}) - \eta(r)]} (1 + \mathcal{O}(h)); \tag{33}$$

- $l \in \text{Jump}$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}} = \Theta(h^{2[a(r_{\max}) + \eta(r_{\max})]}). \tag{34}$$

Now we can estimate the reconstruction error as

$$\begin{aligned}
|u(\mathbf{x}) - \mathcal{R}(\mathbf{x})| &= \left| \sum_l \tilde{\omega}_l (u(\mathbf{x}) - Q_l(\mathbf{x})) \right| \\
&\leq \sum_{l \in \text{Smooth}} \tilde{\omega}_l |u(\mathbf{x}) - Q_l(\mathbf{x})| + \sum_{l \in \text{Jump}} \tilde{\omega}_l |u(\mathbf{x}) - Q_l(\mathbf{x})| \\
&\leq \sum_r \sum_{l \in \text{Smooth}_r} \Theta(h^{2[a(r_{\max} - r) + (\eta(r_{\max}) - \eta(r))]} \mathcal{O}(h^r) \\
&\quad + \sum_{l \in \text{Jump}} \Theta(h^{2(ar_{\max} + \eta(r_{\max}))} \mathcal{O}(1) \\
&= \mathcal{O}(h^{r_{\max}})
\end{aligned} \tag{35}$$

Combining everything above and (20), we arrive at the formal error estimate of the reconstruction.

Lemma 0.1. *Let $\mathcal{R}(\mathbf{x})$ be the reconstruction. Assuming a quasi-uniform mesh, $h_0 = \Theta(h)$, there exists a constant $C > 0$ such that for any $\mathbf{x} \in E_0$,*

$$|u(\mathbf{x}) - \mathcal{R}(\mathbf{x})| \leq Ch^{r_{\max}}, \tag{36}$$

where $r_{\max}$ is defined by (25).

0.2.6 Time stepping

In this section, we briefly talk about suitable time integration technique. We will focus on explicit scheme for two reasons

- Phase package cannot be evaluated with long time intervals.
- Jacobian is hard to evaluate since a MFEM solver is involved.

While forward Euler is sufficient in stability analysis, it only provides first order accuracy. Therefore, we use strong-stability preservation (SSP) scheme proposed by Gottlieb et al. (2001), which is a higher order time stepping scheme while preserving the stability properties of forward Euler. We start with first order forward Euler as

$$u^{n+1} = u^n + \Delta t \tilde{\mathbf{F}}(u^n), \quad (37)$$

where $\tilde{\mathbf{F}}(u^n)$ represents a spatial discretization.

We state a multi-stage SSP Runge-Kutta method as

$$\begin{aligned} u^0 &= u^n, \\ u^{(i)} &= \sum_{j=0}^{i-1} \alpha_{i,j} u^{(j)} + \Delta t \beta_{i,j} \tilde{\mathbf{F}}(u^{(j)}), \quad 1 \leq i \leq m \\ u^{n+1} &= u^m, \end{aligned} \quad (38)$$

where $\sum_j^{i-1} \alpha_{i,j} = 1$ for consistency. In our computation, we have second order accuracy in MFEM solver, and a MLWENO(3,2) reconstruction in FVM. Hence the highest order of time stepping accuracy is three. We write second and third order SSP Runge-Kutta explicit as

$$\begin{aligned} u^0 &= u^n \\ u^{(1)} &= u^{(0)} + \Delta t \tilde{\mathbf{F}}(u^{(0)}) \\ u^{n+1} &= u^{(0)} + \frac{1}{2} \Delta t \tilde{\mathbf{F}}(u^{(0)}) + \frac{1}{2} \Delta t \tilde{\mathbf{F}}(u^{(1)}), \end{aligned} \quad (39)$$

and

$$\begin{aligned} u^0 &= u^n, \\ u^{(1)} &= u^{(0)} + \Delta t \tilde{\mathbf{F}}(u^{(0)}), \\ u^{(2)} &= \frac{3}{4} u^n + \frac{1}{4} u^{(1)} + \frac{1}{4} \Delta t \tilde{\mathbf{F}}(u^{(1)}) \\ u^{n+1} &= \frac{1}{3} u^{(0)} + \frac{2}{3} u^{(2)} + \frac{2}{3} \Delta t \tilde{\mathbf{F}}(u^{(2)}). \end{aligned} \quad (40)$$

0.3 Handling Boundary Conditions

We couple MFEM and FVM solver on a single mesh in our simulation. Therefore, accurate and compatible boundary conditions are assigned accordingly. However, it is usually difficult to manage boundary conditions for higher order finite volume and finite difference method, which require wide stencils.

Some methods address this difficulty by using ghost cells or ghost points outside the computational domain, allowing the use of interior-like stencils near boundaries. However, the extrapolation or estimation of the manufactured values of ghost cells or ghost points are not straightforward. A notable method, the inverse Lax-Wendroff procedure, iteratively substitute normal derivatives of the solution with tangential and time derivatives using the PDE relation to impose more precise values for the ghost cells or ghost points.

We focus on utilizing the flexibility of the ML-WENO reconstruction to tackle the difficulties in a direct way without the use of ghost cells. This approach is independent of the complexity of the governing partial differential equations and intricacies of the geometry of the boundary. Similar discussion using the adaptive order reconstruction can be found in Semplice and Visconti (2020) and Semplice et al. (2023).

In Finite Volume Methods, we must use the boundary condition to determine the numerical flux on the boundary. To assign proper boundary conditions to a FVM discretization, it is necessary to first identify whether the flow is inflow or outflow first. Let u_B denote the value of u on the boundary. We classify the types of boundary conditions based on the magnitude and direction of the numerical flux $\mathbf{f}(u_B) \cdot \boldsymbol{\nu}$.

- If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} > 0$, it is classified as an *outflow* boundary, and no fixed boundary condition should be assigned. Instead a *free outflow* boundary condition is used, where the numerical flux is simply given by $\mathbf{f}(u_B) \cdot \boldsymbol{\nu}$. The outflow boundary may lead to degradation in the accuracy of the numerical solution, as high-order stencils can extend beyond the computational domain. ML-WENO offers the flexibility to add stencils of the desired size that span back into the domain, thereby preserving accuracy near outflow boundaries.
- If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} < 0$, it is classified as an *inflow* boundary. We can understand it as prescribing the inflow flux $\mathbf{f}_B$ or fixing the boundary value u_B as an equivalent to Dirichlet boundary in the context of Finite Element Method. In the inflow case, discontinuous shock may enter the domain, requiring a swift response inside the domain to align with the specified Dirichlet value. To accommodate such potential discontinuities, we can incorporate an extra constant stencil polynomial to the inflow boundary.

- If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} = 0$, it is usually referred to as a *noflow* boundary. No additional treatment on the boundary is required, as the value is fixed at zero.

In the following subsections, we present two examples: the first illustrates a free outflow boundary condition, while the second addresss an inflow Dirichlet boundary condition that generates an incoming shock wave.

0.3.1 Outflow at the boundary

For the outflow example, we test linear transport of a sine wave traveling diagonally across the domain, which takes the true form of solution as

$$u(\mathbf{x}, t) = \{\sin \quad (41)$$

0.3.2 Inflow at the boundary

0.4 Numerical Test

For completeness, we test our ML-WENO algorithm with simple Riemann shock and rarefaction. Let $\mathbf{f}(u) = (u^2/2, u^2/2)$ be the Burgers flux function. In this example, we take the initial condition $u_0(x, y) = 1$, and the computational domain $\Omega = [0, 3] \times [0, 1]$. The true solution is

$$u(x, y, t) = \begin{cases} 0, & x < 0.5, x \geq 0.5t + 1.5, \\ (x - 0.5)/t, & 0.5 \leq x < t + 0.5, \\ 1, & t + 0.5 \leq x < 0.5t + 1.5. \end{cases} \quad (42)$$

We forward marching to the time = 2.0, when trailing rarefaction reaches the leading shock. We use SSP2RK time stepping scheme.

Chapter 1: Code Implementation

In this project, all numerical experiments and simulations are implemented in C++/C, with basic iterative solver and simple parallel data management facilitated by PETSc. In this section, we describe the implementation of a coupled FEM-FVM code, designed to preserve generality and support future optimization and extensibility.

We first present a general implementation for unstructured mesh, followed by a detailed discussion of a specific application involving a logically rectangular mesh. All applications are considered in a two-dimensional setting.

1.1 Geometry

A fundamental step in developing a FEM or FVM code is the indexing of geometric entities. The mesh index data structure implemented in the code is inspired by the classical viewpoint of Boundary Representation (BREP) framework, originally introduced by Baumgart (1975). In the two-dimensional case, each element is bounded by a set of edges, and each edge is defined by a set of vertices. We begin by assuming the following representation for global geometric object:

$$\begin{aligned} \text{Vertex } V_i, \quad i \in ID_V, \\ \text{Edge } E_i, \quad i \in ID_E, \\ \text{Cell } C_i, \quad i \in ID_C, \end{aligned} \tag{1.1}$$

where ID_V , ID_E and ID_C denote countable sets containing all unique global indices for vertices, edges and cells (or faces), respectively. Superscripts are used to indicate subsets of these global index sets. For example, $ID_V^{E_i} \subset ID_V$ represents the set of global vertex indices associated with edge E_i , which is a subset of the global vertex index set.

We assume the following representation for local geometric entities:

$$\begin{aligned}
\text{Vertex} \quad v_i, \quad i \in id_v, \\
\text{Edge} \quad e_i, \quad i \in id_e, \\
\text{Cell} \quad c_i, \quad i \in id_c
\end{aligned} \tag{1.2}$$

where id_v , id_e and id_c denote countable sets containing local index for vertices, edges and faces corresponding to a given geometrical object. Superscripts are used to indicate the target geometrical object. For example, $id_v^{E_i} = \{0, 1\}$ represents the local indices of the two vertices bounding the edge E_i .

We establish a one-to-one mapping between local and global indices through a local-to-global map, defined as follow:

$$f : id \rightarrow ID. \tag{1.3}$$

In practice, it is often unnecessary to explicitly define a local-to-global mapping. Instead, for each local array containing global indices, the array indices naturally represent the local index set. This array-based representation enables the implicit construction of a global-to-local mapping within each geometric entity.

1.1.1 General Unstructured Mesh

In both FEM and FVM solvers, where a cell(element) serves as the fundamental geometry entity. we start with a brief analysis of the information each cell should access from its local perspective. Each polygon cell C_i should have the following access to other geometric objects as

- Adjacent polygons connected by edges;
- Adjacent polygons connected with vertices, but not sharing an edge;
- Incident vertices;
- Edges that bounds this element.

Within the local set, we can relate edges and vertices directly. For convinence, each edge j should has links to:

- Two polygons that sharing this edge;
- Two vertices that bounds this edge.

In FEM and FVM computation, we work with a orientable mesh, where "clock-wise" and "counterclockwise" can be defined consistently. Hence, we can use half edge data structure to regulate direction in a unstructured polygon mesh. In figure 1.1, we

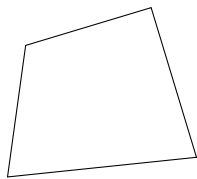


Figure 1.1: An illustration of the half-edge data structure

present an illustration of the half edge data structure applied to a quadrilateral mesh. For each target cell E_i , it sees the vertex V_i and the local link l_i to the next vertex V_i^+ . Hence the direction of the link $l_i : V_i \rightarrow V_i^+$ is determined from the perspective of the target cell C_i .

1.2 Logically Rectangular Mesh

Mesh generation is not the main focus of this project, hence we adopt a much simpler way of implementing quadrilateral mesh, the logically rectangular way. We generate the quadrilateral mesh by randomly distorting each interior vertex coordinates. There is no affine mapping between reference square mesh and generated mesh. The indexing of geometrical objects remain untouched through the vertex distortion.

In our case, all the indices are represented with cartesian coordinates. Therefore, we can setup local to global mapping of geometric objects easily with linear functions. Consider a mesh with $M \times N$ cells, the total number of vertex is $(M +$

$1) \times (N + 1)$ and the total number of edges is $M \times (N + 1) + (M + 1) \times N$. We count horizontal edges first, and the vertical edges are counted starting from $M \times (N + 1)$.

Before we dig into index convention of a this kind of mesh deeply, we introduce a reshape function communicating N-dimensional data and One-dimensional data.

1.2.1 Reshape N-dimensional Data

The natural representation of a 2D cartesian data is a pair of index (i, j) . However, we usually reshape the 2D data into a 1D array for a more natural representation in computation. We give a more general reshape function that projects N-dimensional data to a one-dimensional data. This reshape function can also be used for tensor data, e.g., base smoothness indicator $??$. There exists a bijective mapping as

$$i \leftrightarrow (i_0, i_1, \dots, i_{N-1}). \quad (1.4)$$

The algorithm 1 illustrates the simple implementation of this reshape function. We

Algorithm 1 N dimensional reshape

Consider a N-dimensional array $\{n_0 \times n_1 \times \dots n_{N-1}\}$.

Let $I = 0$ be the result, and $a = 0$ be the multiplier.

Consider an input N-dimensional index $\{i_0, i_1, \dots, i_{N-1}\}$.

1: **for** rank $r = 0, 1, \dots, N - 1$ **do**

2: $I = I + i_r * a$.

3: $a = a \times n_r$.

4: **end for**

can also reverse the process reshaping 1D data back into N-dimensional data with modulo and integer division operation. We use these two index interchangeably. We introduce a new notation highlighting the relation between N-dimensional and one-dimensional data as

$$i(i_0, i_1, \dots, i_{N-1})_{n_0, n_1, \dots, n_{N-1}}, \quad (1.5)$$

where i is the index in one-dimensional array, $(i_0, i_1, \dots, i_{N-1})$ is the N-dimensional index, and $n_0, n_1, \dots, n_{N-1}$ represents the size of N-dimensional array.

1.2.2 Local Vertex Index

We represent local vertex index with the new notation. For a given vertex with local index $i \in id_v$ associated with a cell with global index $I \in ID_C$, we define an index convention as

$$a(a_0, a_1)_{2,2}, \quad (1.6)$$

where $a_0, a_1 \in \{0, 1\}$. We found the next point simply by adding 1 to the one-dimensional index as

$$b \equiv a + 1 \pmod{4}, \quad (1.7)$$

where the direction is naturally defined by pointing from vertex v_a to v_b .

1.2.3 Local Global Index convention

Assuming local vertex and edge number being counted in the same way as indicated in the mesh ??, where the counting starts from left bottom corner. We define the local to global mapping with the local index of vertex and global index of cell. Consider mapping the local index i of vertex v_i in the cell $C_{K(I_0, I_1)_{M, N}}$ as

$$id_v \ni i(i_0, i_1)_{m, n} \rightarrow I(I_0, I_1)_{M+1, N+1} \in ID_V. \quad (1.8)$$

We write the mapped global index with the reshape notation (1.5) as

$$ID_V \ni I(I_0 + i_0, I_1 + i_1)_{M+1, N+1}. \quad (1.9)$$

Note that the bottom global index of left corner vertex share the same N-dimensional index as the global index of the cell. We then discuss the local to global mapping for vertical and horizontal edges. Consider the starting vertex $id_v \ni i(i_0, i_1)$ of the edge e_i such that

- Horizontal Edge;

$$ID_E \ni I(I_0, I_1 + i_1)_{M, N+1}. \quad (1.10)$$

- Vertical Edge.

$$ID_E \ni I(I_0 + i_0, I_1)_{M+1, N} + M \times (N + 1). \quad (1.11)$$

1.3 Dofs and MFEM assembly

In FEM and FVM computation, we need to associate geometrical objects with degrees of freedom. For FVM scheme, the dofs associated with edge flux are straightforward. However, the MFEM dofs require careful treatment, especially for those dofs on the boundary. We follow the idea in Hughes (1987), we need three data processing arrays: ID, IEN and LM arrays. We start with the introduction of these three arrays as

- IEN array.

$$IEN(a, e) = A, \quad (1.12)$$

where a is the local dof index, e is the global element number and A is the global dof index.

- ID array.

$$ID(A) = \begin{cases} P, & A \notin \text{Essen}, \\ 0, & A \in \text{Essen}, \end{cases} \quad (1.13)$$

where P is the position used in global matrix assembly, and Essen denotes the boundary assigned with essential boundary condition.

- LM array.

$$LM = ID(IEN(a, e)). \quad (1.14)$$

In actual code implementation, we create LM array by screening all dofs prior to computation, while IEN and ID array are not actually stored.

1.3.1 Index Convention Example

For $H(\text{div})$ element, the dofs are mainly associated with edge like FVM, which is easy to comprehend and implement. Therefore, we chose to give a detailed description of how Modified Bernardi-Raugel is implemented with the concept of ID, IEN and LM arrays, where vertex dofs and edge dofs are mixed. In Figure 1.2, we illustrate dofs in a bernardi regael element, where two dofs (x dof and y dof) are associated with vertex nodes and one dof (flux dof) associated with edge nodes. We count x dof first y dof second, flux across horizontal edges third and the flux across vertical edges.

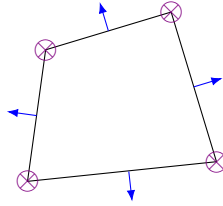


Figure 1.2: An exhibition of dofs associated with a modified Bernardi-Regual element.

In Figure 1.3, we show a simple 4×4 mesh with corresponding index associated with each dof marked. The total dof counts equal to $2 \times (3 \times 3) + 2 \times 3 + 3 \times 2 = 30$. For a column upwelling problem, We assume the following boundary condition is prescribed.

- Bottom Edge;
 - x dof fixed with zero;
 - y dof fixed with a constant upwelling speed;
 - normal flux dof fixed with a constant influx.
- Left and Right side Edges;
 - x dof fixed with zero;

- y dof is free, given natural boundary condition (zero stress);
- normal flux dof fixed with a zero flux.
- Top Edge;
 - x dof fixed with zero;
 - y dof is free, given natural boundary condition (zero stress);
 - normal flux dof is free.

We mark all the dof index that are not prescribed, i.e., will be determined by solving a linear system with superscript a *. These "free" dofs consist of interior dofs and boundary dofs that are given natural boundary condition. We can form a

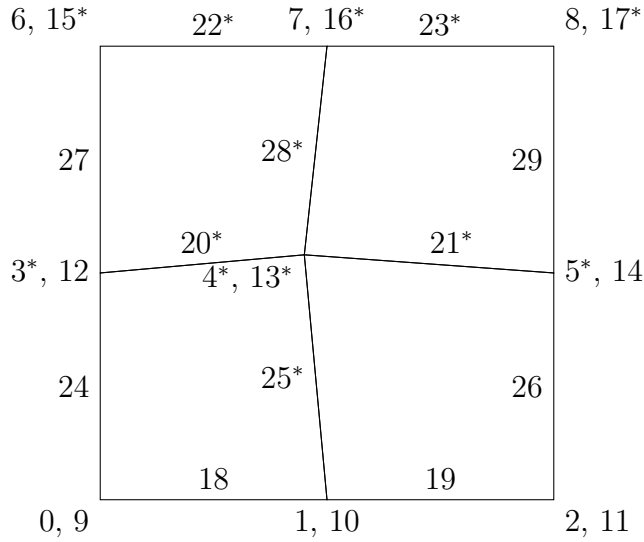


Figure 1.3: An exhibition of br dofs index with a 4×4 mesh.

reduced ID array with the information in Figure 1.3 as The reduced linear system

P	0	1	2	3	4	5	6	7	8	9	10	11
A	3	4	4	13	14	16	20	21	22	23	25	28

Table 1.1: Generated ID array relating to Figure 1.3.

excluding essential boundary dofs will be 12×12 .

1.4 Parallel Implementation

In this section, we discuss some ideas regarding parallel implementation of MFEM and FVM solvers, and present some preliminary timing results for each solver. The parallel implementation of the MFEM and FVM need different treatments.

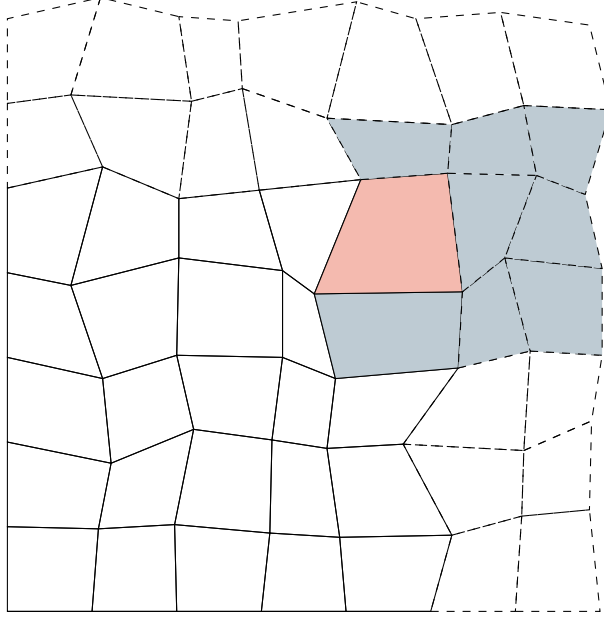


Figure 1.4: An exhibition of ghost region in parallel implementation of ML-WENO scheme.

For FVM discretization, the high order stencils posed some difficulties on parallel implementation. We need to extend from one processor to the adjacent processor with the so-called ghost region, which results in more communication during computation. Consider a ML-WENO(3,2) scheme, as shown in Figure 1.4, we show distributed ghost region with dashed lines. We need to determine the flux across the edge of the red cell. Therefore, a maximum 3×3 stencil should be attached to the outside cell. Hence, in a ML-WENO(3,2) scheme, a two-cell ghost layer should be distributed and updated everytime computing flux.

For MFEM discretization, we encounter different problems. The MFEM discretization, unlike FVM scheme, forms a sparse matrix. The sparse matrix are split

across the processors. The dof in solution vector should be distributed accordingly. Therefore, everytime the solution is being reconstructed, communication between processors will be induced.

We provide strong scaling plot for our test parallel running. We run our MFEM and FVM code with 1,2,4,6 processors

1.4.1 Parallel Index Convention

Index convention in parallel computing requires another "local" index kept within each processor. We can bypass such difficulty by using Hash table data structure that directly mapping to global index with out adding another "local" index. However, in the worst case Hash table suffers $O(n)$ time complexity in searching.

1.5 PETSc Implementation

PETSc, the Protable, Extensible Toolkit for Scientific Computation has been gaining popularity in scientific applications modelled by partial differential equations Balay et al. (2025b) and Balay et al. (2025a). We used three main features from PETSc package.

- Vector (Vec) and matrix (Mat) data structure.
- Data management object (DM).
- Standard linear solver (KSP).

We store global data in Vec and Mat data structure provided by PETSc. Vec and Mat objects in PETSc can complete simple add, dot operations without distinguish between sequential or parallel.

Note that PETSc provides a built-in data management object DMStag. However, we use a more generic data management object DMDA for structured mesh

because we formed reduced linear system in MFEM computation. DMStag, on the other hand, does not provide a straightforward function that can create reduced system for us.

We use the built in KSP solver, e.g., KSPMINRES and KSPCG solver for our application when we solve large sparse system. However, for small system such as 4×4 system solved in ML-WENO stencil polynomial computation, we call lapack directly. We also use provided preconditioner.

Works Cited

Todd Arbogast, Chieh-Sen Huang, and Chenyu Tian. A finite volume multilevel weno scheme for multidimensional scalar conservation laws. *Comput. Methods Appl. Mech. Engrg.*, 421, 2024.

Todd Arbogast, Chieh-Sen Huang, Chenyu Tian, and Gabirel M.Gray. An efficient and accurate approximation to the classic polynomial smoothness indicator. *Submitted.*, 2025.

Satish Balay, Shrirang Abhyankar, Mark F. Adams, Steven Benson, Jed Brown, Peter Brune, Kris Buschelman, Emil Constantinescu, Lisandro Dalcin, Alp Dener, Victor Eijkhout, Jacob Faibussowitsch, William D. Gropp, Václav Hapla, Tobin Isaac, Pierre Jolivet, Dmitry Karpeev, Dinesh Kaushik, Matthew G. Knepley, Fande Kong, Scott Kruger, Dave A. May, Lois Curfman McInnes, Richard Tran Mills, Lawrence Mitchell, Todd Munson, Jose E. Roman, Karl Rupp, Patrick Sanan, Jason Sarich, Barry F. Smith, Hansol Suh, Stefano Zampini, Hong Zhang, Hong Zhang, and Junchao Zhang. PETSc/TAO users manual. Technical Report ANL-21/39 - Revision 3.23, Argonne National Laboratory, 2025a.

Satish Balay, Shrirang Abhyankar, Mark F. Adams, Steven Benson, Jed Brown, Peter Brune, Kris Buschelman, Emil M. Constantinescu, Lisandro Dalcin, Alp Dener, Victor Eijkhout, Jacob Faibussowitsch, William D. Gropp, Václav Hapla, Tobin Isaac, Pierre Jolivet, Dmitry Karpeev, Dinesh Kaushik, Matthew G. Knepley, Fande Kong, Scott Kruger, Dave A. May, Lois Curfman McInnes, Richard Tran Mills, Lawrence Mitchell, Todd Munson, Jose E. Roman, Karl Rupp, Patrick Sanan, Jason Sarich, Barry F. Smith, Stefano Zampini, Hong Zhang, Hong Zhang, and Junchao Zhang. PETSc Web page. <https://petsc.org/>, 2025b. URL <https://petsc.org/>.

Bruce G. Baumgart. A polyhedron representation for computer vision. In *Proceedings of the May 19-22, 1975, National Computer Conference and Exposition*, AFIPS '75, page 589–596, New York, NY, USA, 1975. Association for Computing Machinery. ISBN 9781450379199. doi: 10.1145/1499949.1500071. URL <https://doi.org/10.1145/1499949.1500071>.

Todd Dupont and Ridgway Scott. Polynomial approximation of functions in Sobolev spaces. *Mathematics of Computation*, 34(150):441–463, 1980.

Oliver Friedrich. Weighted essentially non-oscillatory schemes for the interpolation of mean values on unstructured grids. *Journal of Computational Physics*, 144(1):194–212, 1998. ISSN 0021-9991. doi: <https://doi.org/10.1006/jcph.1998.5988>. URL <https://www.sciencedirect.com/science/article/pii/S0021999198959885>.

Sigal Gottlieb, Chi-Wang Shu, and Eitan Tadmor. Strong stability-preserving high-order time discretization methods. *SIAM Review*, 43(1):89–112, 2001. doi: 10.1137/S003614450036757X. URL <https://doi.org/10.1137/S003614450036757X>.

Chieh-Sen Huang, Todd Arbogast, and Chenyu Tian. Multidimensional weno-ao reconstructions using a simplified smoothness indicator and applications to conservation laws. *J. Sci. Comput.*, 97, 2023.

Thomas J.R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, 1987. ISBN 9780486411811.

Guang-Shan Jiang and Chi-Wang Shu. Efficient implementation of weighted eno schemes. *J. Comput. Phys.*, 126:202–228, 1996.

Frank Olver. *Asymptotics and special functions*. AK Peters/CRC Press, 1997.

M. Semplice and G. Visconti. Efficient implementation of adaptive order reconstructions. *Journal of Scientific Computing*, 83, 2020.

M. Semplice, E. Travaglia, and G. Puppo. One-and multi-dimensional cwenoz reconstructions for implementing boundary conditions without ghost cells. *Communications on Applied Mathematics and Computation*, 5:143–169, 2023.